

Caso 1

Arquitectura del Computador y Representación de Datos

Objetivos:

- Usar lenguaje ensamblador para escribir una función de baja complejidad. Esto permitirá reconocer que una instrucción en un lenguaje de alto nivel puede implicar múltiples instrucciones de máquina y afectar el desempeño de un programa, especialmente cuando se ejecuta repetidamente dentro de un ciclo.
- Comprender la estructura y el funcionamiento de la pila para el paso de parámetros y el manejo de variables locales. Esto permitirá analizar el costo asociado a la invocación de funciones o métodos durante la ejecución de un programa.
- Comprender los fundamentos de la representación binaria de la información, tanto sin cifrar como cifrada. Esto permitirá evidenciar que una misma secuencia de ceros y unos puede interpretarse de distintas maneras y que, por tanto, identificar la representación adecuada es fundamental.

MUY IMPORTANTE: Este caso está diseñado para apoyar su proceso de aprendizaje, ayudarle a identificar y resolver dudas antes del parcial, y fortalecer su comprensión de los temas estudiados. El entregable debe reflejar el trabajo, las decisiones y la comprensión del grupo. Por esta razón, **les recomendamos expresamente no usar herramientas de IA generativa para desarrollar el caso:** el propósito central de la actividad es que ustedes realicen el análisis, construyan la solución y puedan explicar las decisiones tomadas. Usar IA generativa para producir la solución puede dar la impresión de haber completado la tarea, pero impedir el desarrollo de las habilidades que serán evaluadas. Si, aun así, deciden apoyarse en estas herramientas, deberán hacerlo de manera crítica, limitada y responsable, sin reemplazar el trabajo propio ni entregar una solución que no comprendan plenamente. Recuerden que desarrollar el caso a consciencia es parte fundamental de la preparación para el parcial, el cual incluirá al menos una pregunta basada en este caso. Las preguntas estarán orientadas a evaluar su comprensión del problema, de la solución desarrollada y de las decisiones tomadas durante su construcción. En consecuencia, apoyarse en una solución que no puedan explicar o justificar puede afectar directamente su desempeño en esa parte de la evaluación.

A. Representación de la información en archivos

En un computador, toda la información —ya sea texto, imagen, audio, video, etc.— se representa como una secuencia de bits. Para facilitar su lectura los bits pueden agruparse en bytes y cada byte puede interpretarse como un entero entre 0 y 255, lo que implica que cualquier tipo de archivo puede interpretarse como un arreglo de enteros.

Esta representación uniforme en bits y bytes permite que los sistemas computacionales procesen distintos tipos de datos mediante los mismos mecanismos básicos de memoria y procesamiento. Sin embargo, en cada contexto se requiere una aplicación capaz de interpretar esas secuencias de forma adecuada. Por ejemplo, un archivo de sonido se almacena como una secuencia de bits, pero para reproducirlo se necesita una aplicación que interprete esos bits como audio.

B. Cifrado de información

El cifrado es un proceso mediante el cual la información se transforma con el fin de ocultar su contenido. A través de un conjunto de reglas y el uso de claves, los datos originales se convierten en una representación diferente que no puede ser interpretada directamente. Desde el punto de vista de representación de la información, cifrar implica modificar los valores que representan la información — generalmente bytes— de manera controlada, de tal forma que solo quien conoce el mecanismo de transformación pueda recuperar el contenido original mediante el proceso inverso, conocido como descifrado.

Cifrado de César y su evolución hacia el cifrado de Vigenère

Uno de los métodos más simples de cifrado es el **cifrado de César**¹. Este algoritmo consiste en desplazar cada elemento de la información una cantidad fija dentro de un conjunto de valores. En su forma clásica, este conjunto corresponde a las letras del alfabeto. Por ejemplo, si se utiliza un desplazamiento de 3 posiciones, la letra 'A' se transforma en 'D', la 'B' en 'E', y así sucesivamente. Cuando se alcanza el final del alfabeto, se usa un comportamiento circular. Así, por ejemplo, usando un desplazamiento de 3, la palabra "ROMA" se transformaría en "URPD".

Desde el punto de vista computacional, este algoritmo puede interpretarse como una operación aritmética sobre la representación numérica de los caracteres. Si los datos se representan como valores enteros, el cifrado de César puede expresarse como una suma seguida de una operación módulo 26 que garantiza que el resultado permanezca dentro del rango de las letras del alfabeto. Sin embargo, este principio no se limita a texto; en un computador, cualquier tipo de información puede verse como una secuencia de bytes, por lo que el mismo concepto puede extenderse a archivos de cualquier tipo utilizando aritmética módulo 256.

El cifrado de César presenta una limitación importante: **utiliza la misma clave para todos los elementos de la información**. Esto lo hace vulnerable a análisis simples. En mensajes de texto, por ejemplo, las letras no aparecen con la misma frecuencia; en español, la letra "e" es muy común, mientras que "k" o "w" aparecen con mucha menor frecuencia. Estas regularidades estadísticas permiten realizar **análisis de frecuencia**. Como el algoritmo aplica el mismo desplazamiento a todas las letras del mensaje, la distribución de frecuencias del texto original se conserva en el texto cifrado, solo que desplazada. Al identificar la letra más frecuente en el texto cifrado y compararla con las frecuencias típicas del idioma, es posible inferir la clave utilizada y recuperar el mensaje original.

Como una evolución de este algoritmo surge el **cifrado de Vigenère**, que introduce el concepto de una clave compuesta. En lugar de aplicar un único desplazamiento constante, el algoritmo utiliza una secuencia de desplazamientos que se repite a lo largo de los datos. Cada elemento del mensaje se transforma utilizando un valor distinto de la clave.

Cada elemento del texto original se combina con un elemento de la clave mediante una operación aritmética (por ejemplo, suma módulo 256 en el caso de bytes) y la clave se recorre de manera cíclica, de modo que cuando se alcanza su final, se vuelve a comenzar desde el inicio. El proceso de descifrado consiste en aplicar la operación inversa, lo que permite recuperar el dato original.

Aunque, al igual que en el cifrado de César, este algoritmo se describe tradicionalmente sobre letras, el mismo principio puede extenderse a la representación binaria de la información realizando operaciones de suma y resta entre bytes, utilizando aritmética módulo 256 en lugar de módulo 26. De esta manera, el algoritmo puede aplicarse no solo a texto, sino a cualquier tipo de archivo, independientemente de su contenido.

La principal ventaja del cifrado de Vigenère frente al de César es que **rompe la uniformidad de la transformación**. Dado que diferentes posiciones del mensaje se transforman de manera distinta, se

¹ El cifrado de César recibe su nombre de Julio César, quien, según relatos históricos, lo utilizaba para proteger comunicaciones militares en la antigua Roma.. Para una descripción histórica más detallada, puede consultarse: Singh, S. (1999). *The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography*. Anchor Books. Un extracto de ese libro sobre el tema puede ser consultado en <https://www.perlego.com/ereader/685842>

reduce significativamente la aparición de patrones repetitivos en la información cifrada. Esto hace que el análisis del mensaje sea considerablemente más complejo y mejora la robustez del algoritmo frente a ataques simples.

Desde la perspectiva de la representación de la información, el cifrado de Vigenère permite evidenciar que la seguridad no depende únicamente de la operación aplicada, sino también de cómo se utilizan los datos auxiliares (en este caso, la clave) para modificar la información original. En un entorno computacional, donde todo se representa como bytes, este algoritmo puede aplicarse de manera uniforme a cualquier tipo de archivo, lo que refuerza la idea fundamental de que **la naturaleza de los datos es irrelevante cuando se trabaja a nivel de su representación binaria**.

Si bien este tipo de cifrado es vulnerable a diferentes técnicas de análisis y no ofrece un nivel de seguridad adecuado para aplicaciones reales, resulta suficientemente ilustrativo para el propósito de este caso, ya que nos permite analizar cómo se representa la información en binario, cómo se puede transformar esa representación, cómo se puede recuperar la representación inicial y cómo implementar dichas transformaciones.

C. Actividades

Esta actividad le permitirá comprender cómo un compilador transforma un programa escrito en C a lenguaje ensamblador. Para ello, se utiliza como ejemplo un programa que procesa archivos como secuencias de bytes, lo que permite hacer explícitos los mecanismos mediante los cuales el procesador manipula datos usando registros, memoria y pila. En particular, se analizará cómo el compilador implementa el manejo de variables locales, el acceso a parámetros y las llamadas entre funciones, aspectos fundamentales para entender la ejecución de programas a bajo nivel.

Adicionalmente, este caso permitirá practicar dos mecanismos ampliamente utilizados para el paso de parámetros a una función: el paso por pila y el paso por registro. El paso por pila ofrece flexibilidad, pues permite manejar un número arbitrario de parámetros y permite llamadas recursivas; sin embargo, implica accesos adicionales a memoria. Por su parte, el paso por registros reduce dichos accesos y mejora el desempeño, pero está limitado por la cantidad de registros disponibles y requiere convenciones específicas para preservar su contenido. Comprender las ventajas y limitaciones de ambos mecanismos permite entender las decisiones que toman los compiladores y las convenciones de llamada empleadas por las arquitecturas modernas.

Junto con este enunciado se entrega un programa en C que implementa el cifrado y descifrado de Vigenère. Usted debe traducir a lenguaje ensamblador las funciones que cifran y descifran bloques de datos leídos desde un archivo: `ajustarRangoByte` y `transformarBytesClave`.

A continuación, se describen las actividades que debe desarrollar:

1. Comprensión del programa

Analice el programa en C suministrado e identifique:

- El funcionamiento general del programa, incluyendo la cadena de llamadas entre funciones.
- Los parámetros y variables locales de las funciones de interés.
- El propósito de las funciones `transformarBytesClave` y `ajustarRangoByte`.

2. Análisis de la pila y de las convenciones de paso de parámetros

- A partir del código C de las funciones, identifique las variables locales y los parámetros.
- Para un ejemplo pequeño, siga manualmente las primeras 4 o 5 iteraciones de `transformarBytesClave` y escriba los valores correspondientes:

i	buffer[i] inicial	índiceClave	clave[índiceClave]	temp antes de ajustar	temp después de ajustar	buffer[i] final
0						
1						
...						

- ¿Qué hace transformarBytesClave y para qué sirve ajustarRangoByte?
- ¿Por qué transformarBytesClave retorna el valor de índiceClave? Explique qué información representa ese valor al terminar la función y qué relación tiene con el recorrido de la clave.
- Analice la función transformarBytesClave, en la cual haremos una traducción haciendo paso de parámetros por pila. Dibuje el estado de la pila justo antes de ejecutar la primera instrucción de la función y justo antes de ejecutar la instrucción de retorno. Identifique la ubicación de cada parámetro y de cada variable local, y justifique los desplazamientos empleados para acceder a ellos.
- Realice el mismo análisis para la función ajustarRangoByte. En esta función practicaremos el paso de parámetros por registro, por lo que deberá cambiar la forma como la función accede al parámetro. Dibuje el estado de la pila justo antes de ejecutar la primera instrucción de la función y justo antes de ejecutar la instrucción de retorno. Compare ambos mecanismos e identifique sus principales diferencias, ventajas y desventajas.

3. Ejecución del programa

- Compile y ejecute el programa. Para ejecutarlo, incluya en la línea de comandos la operación que se va a realizar, el archivo de entrada, el archivo de salida y la clave. Realice pruebas con archivos de distintos tipos, por ejemplo: un documento Word, un archivo PDF, una imagen y un archivo de audio. En cada caso, cifre el archivo usando una clave, intente abrir el archivo cifrado con la aplicación correspondiente y luego descifrelo usando la misma clave. Diligencie una fila de la siguiente tabla por cada archivo probado. Además, averigüe qué es SHA-256 y por qué es importante usarlo en el contexto de estas pruebas.

Tipo	Línea de comando usada	Tamaño original	Tamaño cifrado	¿abre cifrado?	Tamaño descifrado	¿abre descifrado?	SHA-256 original ²	SHA-256 descifrado
Word								
PDF								
Imagen								
Audio								

- Usando un editor de texto o un visor hexadecimal, revise el contenido del archivo cifrado. Observe que, aunque el archivo sigue siendo una secuencia de bytes, normalmente deja de ser interpretable por la aplicación que abrió el archivo original. Para cada uno de los cuatro archivos, incluya en el informe los primeros 16 bytes del archivo original, del archivo cifrado y del archivo descifrado, representados en hexadecimal.

- Ejecute el programa en modo de descifrar tomando como entrada el archivo resultante del proceso de cifrado, como salida un archivo con la extensión correspondiente al tipo de archivo original y la misma clave usada para el cifrado. Verifique que el archivo descifrado pueda abrirse correctamente y conserve su contenido original.
Responda las siguientes preguntas en su informe. En sus respuestas, utilice el concepto de representación de la información trabajado en este caso y sustente sus conclusiones a partir de las pruebas realizadas. No se limite a describir el resultado observado; explique las razones de dicho comportamiento.
- a. ¿El tamaño del archivo cambia al cifrarlo? Explique el resultado a partir del algoritmo utilizado.
 - b. Un archivo cifrado puede conservar exactamente el mismo nombre y extensión que el original y, sin embargo, la aplicación correspondiente puede no reconocerlo. Explique por qué.
 - c. Compare los primeros 16 bytes del archivo original, cifrado y descifrado. ¿Qué evidencia proporcionan sobre lo que hace el algoritmo y sobre el proceso de descifrado?
 - d. Los cuatro archivos probados tienen estructuras y contenidos muy diferentes. ¿Qué parte del comportamiento del programa depende del tipo de archivo y qué parte es completamente independiente de él? Justifique a partir de sus pruebas.
 - e. Seleccione uno de los archivos cifrados y trate de descifrarlo utilizando una clave distinta de la empleada durante el cifrado. ¿El programa termina con error o sin error? Explique por qué. ¿Cómo podría demostrar objetivamente que el resultado del descifrado es incorrecto?

4. Traducción a Ensamblador

- Traduzca a ensamblador las dos funciones indicadas. La traducción debe seguir el algoritmo planteado en la versión en C. No optimice ni modifique la lógica del programa; traduzca únicamente el código entregado.
- Limite la traducción a las funciones solicitadas; no modifique el resto del programa.
- Las variables locales deben manejarse en la pila. No use los registros con este fin. En transformarBytesClave para referirse a parámetros o variables locales debe usar desplazamientos en la pila. Para ajustarRangoByte, tenga en cuenta que el parámetro pasa por registro.
- Incluya comentarios para las instrucciones en ensamblador.

5. Declaración sobre uso de Inteligencia Artificial

Aunque recomendamos expresamente no usar herramientas de IA generativa para desarrollar el caso, el grupo **debe declarar de manera explícita** en su informe si las utilizó o no durante el desarrollo del trabajo. Esta declaración hace parte de la transparencia académica esperada en la elaboración del caso.

- **Sin uso de IA:** El informe debe señalar explícitamente: *“declaramos que no utilizamos herramientas de IA generativa para comprender el caso, desarrollar o depurar el código, realizar los análisis solicitados o elaborar las respuestas del informe”*. La omisión deliberada del uso de herramientas de IA generativa, cuando estas hayan sido empleadas en cualquiera de las actividades del caso, **se considerará una declaración falsa** sobre la autoría y el proceso de elaboración del trabajo y, por tanto, constituirá fraude académico, sujeto a las disposiciones institucionales correspondientes.
- **Con uso de IA:** El informe debe señalar explícitamente: *“declaramos que utilizamos herramientas de IA generativa durante el desarrollo del caso y presentamos en el informe el*

análisis crítico sobre su uso". Este análisis debe incluir una explicación de para qué actividades se utilizó, qué tipo de consultas se realizaron y qué respuestas se obtuvieron; debe evaluar la pertinencia y corrección de dichas respuestas, identificando tanto aportes útiles como errores, imprecisiones o limitaciones; debe describir qué modificaciones realizó el grupo sobre las respuestas o soluciones propuestas por la IA y cómo se verificó que el resultado final fuera correcto. También debe analizar en qué medida el uso de la herramienta contribuyó —o no— a la comprensión del programa en C, de la traducción a ensamblador y de los conceptos estudiados en el caso. Debe incluir ejemplos concretos de las interacciones más relevantes para sustentar el análisis. Si se encuentran diferencias en sintaxis, estructura, mecanismo de paso de parámetros o, en general, con la manera como se presentó el tema en clase, se deben identificar estas diferencias y su impacto. Si se hace uso de instrucciones que no se hayan visto en clase, se debe explicar su funcionamiento y la manera como estas contribuyen a la solución presentada.

E. Recomendaciones

Al momento de hacer la traducción a ensamblador dentro del programa en C, tenga en cuenta las siguientes recomendaciones:

- No modifique las estructuras de datos ni el main del programa.
- Compile su programa usando el siguiente comando:

```
clang -m32 -masm=intel -fms-extensions -O0 -fno-pic -fno-unwind-tables -fno-asynchronous-unwind-tables -fno-omit-frame-pointer -fno-stack-protector vigenere.c -o vigenere
```

- Primero haga la conversión de la función `ajustarRangoByte`. Use el nombre simbólico para referirse al parámetro `valor` y permita que el retorno se haga en C. Pruebe el programa y verifique que el comportamiento es el esperado.
- Realice la traducción de `transformarBytesClave`. Como esta función se encuentra dentro de un programa en C, el compilador normalmente se encarga de guardar `ebp` y de manejar el protocolo de salida. Sin embargo, en este caso se quiere que usted implemente explícitamente el manejo de la pila. Por ello, es necesario indicarle al compilador que no genere automáticamente esas instrucciones. Para hacerlo, declare el encabezado de la función de la siguiente manera:

```
declspec(naked) int transformarBytesClave(unsigned char *buffer,
                                         int longitud,
                                         unsigned char *clave,
                                         int tamClave,
                                         int indiceClave,
                                         int modo)
```

- Haga la traducción de `ajustarRangoByte` teniendo en cuenta las mismas consideraciones que en la traducción de la función anterior: se debe declarar el encabezado de la función usando `declspec(naked)` para que el compilador no genere instrucciones para manejo de la pila. Adicionalmente, modifique la función para recibir el parámetro por registro y para incluir las instrucciones ensamblador que implementan el retorno.
- En cuanto a la salvaguarda de registros, puede necesitarla o no; depende de cómo sea su programa.

F. Condiciones de entrega

- En un archivo .zip entregue el código fuente del programa y un informe con el nombre de los integrantes. El nombre del archivo debe ser: caso1_login1_login2.zip
- El trabajo se realiza en grupos de 2 estudiantes. No debe haber consultas entre grupos.

- El grupo responde solidariamente por todo el contenido del trabajo. La elaboración debe ser conjunta; no se considera trabajo en grupo dividir puntos o desarrollar partes independientes sin comprensión compartida.
- En el parcial se incluirá una pregunta sobre el caso.
- El proyecto debe ser entregado por Bloque Neón por uno solo de los integrantes del grupo. **Al comienzo del documento PDF deben aparecer los nombres y códigos de estudiante de todos los integrantes.** Si un integrante no aparece en el documento entregado, el grupo podrá informarlo posteriormente; sin embargo, habrá una penalización: la calificación asignada será distribuida de forma equitativa entre los integrantes del grupo.
- Tenga en cuenta las reglas establecidas en el programa del curso sobre la obligatoriedad del trabajo en grupo.
- **Se debe entregar por Bloque Neón a más tardar el 28 de agosto a las 11:59 p.m.**
- **La fecha de entrega no será movida.**
- **Política de entrega tarde.** Para las entregas tarde, se aplicará la siguiente política: por cada 30 minutos o fracción de retraso, con respecto a la hora de entrega establecida en Bloque Neón, habrá una penalización de 0,5/5.

G. Entregables

- Código del programa en C: con los fragmentos de código traducidos a ensamblador.
 - El código traducido debe incluir comentarios.
 - Incluya, al comienzo del programa C modificado, un comentario con el nombre y código de los autores de la modificación.
 - El programa debe compilar y correr en la plataforma indicada (haga las pruebas necesarias antes de la entrega).
- Un informe en formato PDF que incluya los nombres y códigos de estudiante de todos los integrantes, la declaración sobre uso de IA, las respuestas a todas las preguntas planteadas y los dibujos de la pila para las funciones traducidas a ensamblador. El informe debe estar debidamente estructurado, presentado de manera profesional y redactado con claridad, evitando entregas apresuradas o carentes de formato.